

Library Management System – Phase 3

Course Name: CSE 3330-004

Instructor: PROFESSOR NADRA GUIZANI

Team Members: FAHAD SSENTONGO, MANAN SHAH, LAM NINH

Date: 29 April 2025

Honor Code

I pledge, on my honor, to uphold UT Arlington's tradition of academic integrity, a tradition that values hard work and honest effort in the pursuit of academic excellence.

I promise that I will submit only work that I personally create or that I contribute to group collaborations, and I will appropriately reference any work from other sources. I will follow the highest standards of integrity and uphold the spirit of the Honor Code.

Task 1:

Query description: Add an extra column 'Late' to the Book_Loan table. Values will be 0-for non-late returns, and 1-for late returns. Then update the 'Late' column with '1' for all records that they

have a return date later than the due date and with '0' for those were returned on time.

Query for Query 1:

```
ALTER TABLE Book_Loans
```

```
ADD Late INT;
```

```
UPDATE Book_Loans
```

```
SET Late = (CASE
```

```
    WHEN (Book_Loans.Returned_date BETWEEN
```

```
    Book_Loans.date_out AND Book_Loans.due_date) THEN 0
```

```
    ELSE 1
```

```
    END);
```

```
SELECT * FROM Book_Loans;
```

book_id	branch_id	card_no	date_out	due_date	returned_date	Late
1	1	123456	2022-01-01	2022-02-01	2022-02-01	0
2	1	789012	2022-01-02	2022-02-02	NULL	1
3	2	345678	2022-01-03	2022-02-03	NULL	1
4	3	901234	2022-01-04	2022-02-04	2022-02-04	0
5	1	567890	2022-01-05	2022-02-05	2022-02-09	1
6	2	234567	2022-01-06	2022-02-06	2022-02-10	1
7	2	890123	2022-01-07	2022-02-07	2022-03-08	1
8	3	456789	2022-01-08	2022-02-08	2022-03-10	1
9	1	111111	2022-01-09	2022-02-09	2022-02-06	0
10	2	222222	2022-01-10	2022-02-10	2022-02-07	0
11	1	333333	2022-03-01	2022-03-08	2022-03-08	0
12	3	444444	2022-03-03	2022-03-10	2022-03-10	0
13	3	555555	2022-02-03	2022-03-03	2022-02-18	0
14	1	565656	2022-01-14	2022-02-14	2022-03-31	1
15	3	676767	2022-01-15	2022-02-15	2022-02-21	1
16	2	787878	2022-03-05	2022-03-12	2022-03-24	1
17	3	989898	2022-03-23	2022-03-30	2022-03-30	0
18	3	121212	2022-01-18	2022-02-18	2022-02-18	0
19	1	232323	2022-03-24	2022-03-31	2022-03-31	0
20	3	343434	2022-01-21	2022-02-21	2022-02-21	0
21	3	454545	2022-01-24	2022-02-24	2022-02-24	0

21 rows in set (0.00 sec)

QUERY 2 Query description: Add an extra column 'LateFee' to the Library_Branch table, decide late fee per day for each branch and update that column.

Query 2:

```
ALTER TABLE Library_Branch
```

```
ADD LateFee DECIMAL(10, 2);
```

```
UPDATE Library_Branch
```

```
SET LateFee = (CASE
```

```
    WHEN (Library_Branch.branch_id = 1) THEN 1.20
```

```
    WHEN (Library_Branch.branch_id = 2) THEN 0.75
```

```
    WHEN (Library_Branch.branch_id = 3) THEN 2.30
```

```
    ELSE 0.50
```

```
    END);
```

```
SELECT * FROM Library_Branch;
```

branch_id	branch_name	branch_address	LateFee
1	Main Branch	123 Main St, New York, NY 10003	1.20
2	West Branch	456 West St, Arizona, AR 70622	0.75
3	East Branch	789 East St, New Jersy, NY 32032	2.30
4	North Branch	456 NW, Irving, TX 76100	0.50
5	UTA Branch	123 Cooper St, Arlington TX 76101	0.50

5 rows in set (0.00 sec)

QUERY 3 Query description: Create a view vBookLoanInfo that retrieves all information per book loan. The view should have the following attributes:

- Card_No,
- Borrower Name
- Date_Out,
- Due_Date,

- Returned_date
- Total Days of book loaned out as 'TotalDays'– you need to change weeks to days
- Book Title
- Number of days later return – if returned before or on due_date place zero
- Branch ID
- Total Late Fee Balance 'LateFeeBalance' – If the book was not returned late than fee = '0'

Query for Query 3 create view:

```
CREATE VIEW vBookLoanInfo
AS SELECT BW.card_no AS Card_No,
BW.name AS [Borrower Name],
B_L.date_out AS Date_Out,
B_L.due_date AS Due_Date,
B_L.Returned_date,
CAST((JulianDay(B_L.Returned_date) - JulianDay(B_L.date_out)) AS INTEGER) AS
TotalDays,
Bk.title AS [Book Title],
CASE
WHEN (B_L.Returned_date BETWEEN B_L.date_out AND B_L.due_date) THEN 0
ELSE CAST((JulianDay(B_L.Returned_date) - JulianDay(B_L.due_date)) AS INTEGER)
END AS [Number of days later return],
L_B.branch_id AS [Branch ID],
CASE WHEN (B_L.Returned_date BETWEEN B_L.date_out AND B_L.due_date) THEN 0
ELSE
L_B.LateFee * CAST((JulianDay(B_L.Returned_date) - JulianDay(B_L.due_date)) AS
INTEGER)
```

END AS LateFeeBalance

FROM Borrower AS BW

JOIN Book_Loans AS B_L ON BW.card_no = B_L.card_no

JOIN Book AS Bk ON B_L.book_id = Bk.book_id

JOIN Library_Branch AS L_B ON B_L.branch_id = L_B.branch_id;

SELECT * FROM vBookLoanInfo;

Card_No	Borrower Name	Date_Out	Due_Date	Returned_date	TotalDays	Book Title	Number of days returned late	Branch ID	LateFeeBalance
123456	John Smith	2022-01-01	2022-02-01	2022-02-01	31	To Kill a Mockingbird	0	1	0
789012	Jane Doe	2022-01-02	2022-02-02	NULL	NULL	1984	NULL	1	NULL
345678	Bob Johnson	2022-01-03	2022-02-03	NULL	NULL	Pride and Prejudice	NULL	2	NULL
901234	Sarah Kim	2022-01-04	2022-02-04	2022-02-04	31	The Great Gatsby	0	3	0
567890	Tom Lee	2022-01-05	2022-02-05	2022-02-09	35	One Hundred Years of Solitude	4	1	4.80
234567	Emily Lee	2022-01-06	2022-02-06	2022-02-10	35	Animal Farm	4	2	3.00
890123	Michael Park	2022-01-07	2022-02-07	2022-03-08	60	The Catcher in the Rye	29	2	21.75
456789	Laura Chen	2022-01-08	2022-02-08	2022-03-10	61	Lord of the Flies	30	3	69.00
111111	Alex Kim	2022-01-09	2022-02-09	2022-02-06	28	Brave New World	0	1	0
222222	Rachel Lee	2022-01-10	2022-02-10	2022-02-07	28	The Picture of Dorian Gray	0	2	0
333333	William Johnson	2022-03-01	2022-03-08	2022-03-08	7	The Alchemist	0	1	0
444444	Ethan Martinez	2022-03-03	2022-03-10	2022-03-10	7	The God of Small Things	0	3	0
555555	Grace Hernandez	2022-02-03	2022-03-03	2022-02-18	15	Wuthering Heights	0	3	0
565656	Sophia Park	2022-01-14	2022-02-14	2022-03-31	76	The Hobbit	45	1	54.00
676767	Olivia Lee	2022-01-15	2022-02-15	2022-02-21	37	The Lord of the Rings	6	3	13.80
787878	Noah Thompson	2022-03-05	2022-03-12	2022-03-24	19	The Hitchhiker's Guide to the Galaxy	12	2	9.00
989898	Olivia Smith	2022-03-23	2022-03-30	2022-03-30	7	The Diary of a Young Girl	0	3	0
121212	Chloe Park	2022-01-18	2022-02-18	2022-02-18	31	The Da Vinci Code	0	3	0
232323	William Chen	2022-03-24	2022-03-31	2022-03-31	7	The Adventures of Huckleberry Finn	0	1	0
343434	Olivia Johnson	2022-01-21	2022-02-21	2022-02-21	31	The Adventures of Tom Sawyer	0	3	0
454545	Dylan Kim	2022-01-24	2022-02-24	2022-02-24	31	A Tale of Two Cities	0	3	0

21 rows in set (0.00 sec)

GUI Interface Queries:

QUERY 1

Query description: User checks out a book, add it to Book_Loan, the number of copies needs to be updated in the Book_Copies table. Show the output of the updated Book_Copies.

Code and Query for Query 1:

```
# retrieve book_id of selected book title
```

```
submit_cursor.execute("SELECT book_id FROM Book WHERE title = ?", (book_title.get(),))
```

```
book_output = submit_cursor.fetchall()
```

```
selected_book_id = book_output[0][0]
```

```
# retrieve branch_id of book_id of selected book title
```

```
submit_cursor.execute("SELECT branch_id FROM Book_Copies WHERE book_id = ?",
```

```
(int(str(selected_book_id)),))
```

```
branch_output = submit_cursor.fetchall()
```

```
branch_id = branch_output[0][0]
```

```
# add new book checkout to Book_Loans
```

```
submit_cursor.execute("INSERT INTO Book_Loans VALUES (:book_id, :branch_id, :card_no,  
:date_out, :due_date, :Returned_date, :Late)",
```

```
{
```

```
'book_id': int(selected_book_id),
```

```
'branch_id': int(branch_id),
```

```
'card_no': int(card_number.get()),
```

```
'date_out': date_out.get(),
```

```
'due_date': due_date.get(),
```

```
'Returned_date': returned_date.get(),
```

```
'Late': 0
```

```
})
```

```
# update Late column of Book_Loans
```

```
submit_cursor.execute("UPDATE Book_Loans SET Late = (CASE WHEN  
(Book_Loans.Returned_date BETWEEN Book_Loans.date_out AND Book_Loans.due_date)  
THEN 0 ELSE 1 END)")
```

```
# update number of copies in Book_Copies
```

```
submit_cursor.execute("UPDATE Book_Copies SET no_of_copies = no_of_copies - 1 WHERE  
book_id = ? AND branch_id = ?", (int(str(selected_book_id)), int(str(branch_id)),))
```

The book_id '3' (Pride and Prejudice) initially had '1' copy at branch_id '2', but its output shows '0' no_of_copies for book_id '3' to show that a copy of the book with book_id '3' has been checked out

The screenshot shows the 'LMS Application' window with a tabbed interface. The 'Q1 (Checkout)' tab is active. The 'Checkout Book' form contains the following fields and values:

- Book Title: 1984
- Borrower Card Number: 121212
- Date Out (YYYY-MM-DD): 2025-04-29
- Due Date (Calculated - 31 days from Date Out): 2025-05-30

A 'Checkout Book' button is present. Below the form, a message states: 'Book '1984' checked out successfully to borrower 121212 from branch 1. Due Date: 2025-05-30'.

The 'SQL Queries Executed:' section displays the following queries:

```
SELECT book_id FROM Book WHERE title = '1984';  
SELECT branch_id FROM Book_Copies WHERE book_id = 2 AND no_of_copies > 0 LIMIT 1;  
INSERT INTO Book_Loans (book_id, branch_id, card_no, date_out, due_date, Returned_date, Late) VALUES (2, 1, 121212, '2025-04-29', '2025-05-30', NULL, 0);  
UPDATE Book_Copies SET no_of_copies = no_of_copies - 1 WHERE book_id = 2 AND branch_id = 1;
```

The 'Book Copies Output:' section shows a table with the following data:

Book ID	Branch ID	No. of Copies
2	1	1

QUERY 2

Query description: Add information about a new Borrower. Do not provide the CardNo in your

query. Output the card number as if you are giving a new library card.

query that your code executes.

Code and Query for Query 2:

add new borrower to Borrower

```
submit_cursor.execute("INSERT INTO Borrower(name, address, phone) VALUES (?, ?, ?)",  
(name.get(), address.get(), phone.get(),))
```

The screenshot shows a web application window titled "LMS Application". It has a tabbed interface with tabs for "Q1 (Checkout)", "Q2 (Add Borrower)", "Q3 (Add Book)", "Q4 (Copies Loaned)", "Q5 (Late Loans by Due Date)", "Q6a (Borrower Late Fees)", and "Q6b (Book Loan Info)". The "Q2 (Add Borrower)" tab is active. The form is titled "Add New Borrower" and has a "Query Purpose: Add information for a new library borrower." Below this, there are three input fields: "Name:" with the value "Test", "Address:" with the value "123 Test", and "Phone:" with the value "682-234-567". There is an "Add Borrower" button. Below the button, it says "New borrower 'Test' added successfully." and "SQL Query:". The SQL query is displayed in a text area: "INSERT INTO Borrower(name, address, phone) VALUES ('Test', '123 Test', '682-234-567');". Below the query, there is a "Result:" section showing "New Library Card Issued:" and "Card Number: 989899".

QUERY 3

Query description: Add a new Book with publisher (use can use a publisher that already exists) and author information to all 5 branches with 5 copies for each branch.

add new book with publisher info

```
submit_cursor.execute("INSERT INTO Book(title, book_publisher) VALUES (?, ?)",  
(book_title.get(), book_publisher.get(),))
```

```
# retrieve book_id of newly added book
```

```
submit_cursor.execute("SELECT book_id FROM Book WHERE title = ? AND book_publisher  
= ?", (book_title.get(), book_publisher.get(),))
```

```
book_output = submit_cursor.fetchall()
```

```
selected_book_id = book_output[0][0]
```

```
# add author info of newly added book
```

```
submit_cursor.execute("INSERT INTO Book_Authors(book_id, author_name) VALUES (?, ?)",  
(int(str(selected_book_id)), author_name.get(),))
```

```
# retrieve number of library branches
```

```
submit_cursor.execute("SELECT COUNT(*) FROM Library_Branch")
```

```
count_of_library_branches = submit_cursor.fetchall()
```

```
num_of_library_branches = int(count_of_library_branches[0][0])
```

```
# add 5 copies per branch for newly added book
```

```
for i in range(1, num_of_library_branches + 1):
```

```
submit_cursor.execute("INSERT INTO Book_Copies(book_id, branch_id, no_of_copies)  
VALUES (?, ?, ?)",
```

```
(int(str(selected_book_id)), i, 5,))
```

LMS Application

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Add New Book

Query Purpose: Add a new book with publisher and author information to all branches. Creates publisher if it doesn't exist.

Book Title:Test Book

Publisher Name:Test Publisher

Author Name:Test Author

Add Book

New book 'Test Book' by Test Author (Publisher: Test Publisher) added successfully with 5 copies at each of the 3 branches.

SQL Queries Executed:

INSERT INTO Publisher (publisher_name) VALUES ('Test Publisher');
INSERT INTO Book(title, book_publisher) VALUES ('Test Book', 'Test Publisher');
INSERT INTO Book_Authors(book_id, author_name) VALUES (22, 'Test Author');
SELECT COUNT(*) FROM Library_Branch;
INSERT INTO Book_Copies(book_id, branch_id, no_of_copies) VALUES (22, 1, 5);
INSERT INTO Book_Copies(book_id, branch_id, no_of_copies) VALUES (22, 2, 5);
INSERT INTO Book_Copies(book_id, branch_id, no_of_copies) VALUES (22, 3, 5);

Book Copies Added:

Branch ID	No. of Copies Added
1	5
2	5
3	5

QUERY 4

Query description: Given a book title list the number of copies loaned out per branch.

Code and Query for Query 4:

```
# add data to copies_loaned table output for selected book title
```

```
copies_loaned_cursor.execute(
```

```
"SELECT Bk.book_id, Bk.title, B_L.branch_id, L_B.branch_name,
```

```
COUNT(*) AS num_of_copies_loaned
```

```
FROM Book AS Bk NATURAL JOIN Book_Loans AS B_L
```

```
NATURAL JOIN Library_Branch AS L_B
```

```
WHERE Bk.title = ? GROUP BY Bk.book_id, Bk.title", (book_title.get(),))
```

Note: In the original Book_Loans table, the book Pride and Prejudice (with book_id '3') originally had one copy loaned out to a borrower, but the output shows '2' no_of_copies_loaned for the book Pride and Prejudice because Query 1 had checked out another copy of the book Pride and Prejudice.

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Copies Loaned per Branch

Query Purpose: List the number of copies loaned out per branch for a given book title.

Book Title:

1984

List Copies Loaned

Copies loaned out for '1984':

SQL Query Executed:

```

SELECT
    Bk.book_id,
    Bk.title,
    B_L.branch_id,
    L_B.branch_name,
    COUNT(*) AS num_of_copies_loaned
FROM Book AS Bk
JOIN Book_Loans AS B_L ON Bk.book_id = B_L.book_id
JOIN Library_Branch AS L_B ON B_L.branch_id = L_B.branch_id

```

Copies Loaned Output:

Book ID	Book Title	Branch ID	Branch Name	Copies Loaned
2	1984	1	Main Branch	2

SQL Query Executed:

```

SELECT
    Bk.book_id,
    Bk.title,
    B_L.branch_id,
    L_B.branch_name,
    COUNT(*) AS num_of_copies_loaned
FROM Book AS Bk
JOIN Book_Loans AS B_L ON Bk.book_id = B_L.book_id
JOIN Library_Branch AS L_B ON B_L.branch_id = L_B.branch_id

```

SQL Query Executed:

```

    Bk.title,
    B_L.branch_id,
    L_B.branch_name,
    COUNT(*) AS num_of_copies_loaned
FROM Book AS Bk
JOIN Book_Loans AS B_L ON Bk.book_id = B_L.book_id
JOIN Library_Branch AS L_B ON B_L.branch_id = L_B.branch_id
WHERE Bk.title = '1984'
GROUP BY Bk.book_id, Bk.title, B_L.branch_id, L_B.branch_name;

```

QUERY 5

Given any due date range list the Book_Loans that were returned late and how many days they were late.

add data to days_late table output

```

days_late_cursor.execute(

"SELECT B_L.book_id, B_L.branch_id, B_L.card_no, B_L.date_out, B_L.due_date,

B_L.Returned_date,

CASE

WHEN (B_L.Returned_date BETWEEN B_L.date_out AND B_L.due_date)

THEN 0

ELSE CAST((JulianDay(B_L.Returned_date) - JulianDay(B_L.due_date))

AS INTEGER) END AS num_of_days_late

FROM Book_Loans AS B_L

WHERE (B_L.due_date BETWEEN ? AND ?) AND (num_of_days_late != 0)",

(first_due_date.get(), second_due_date.get(),))

```

LMS Application

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Late Loans by Due Date Range

Query Purpose: List book loans returned late within a specified due date range.

Start Due Date (YYYY-MM-DD): 1000-10-10

End Due Date (YYYY-MM-DD): 2025-04-29

List Late Loans

Late loans with due dates between 1000-10-10 and 2025-04-29:

SQL Query Executed:

```

SELECT
    B_L.book_id,
    B_L.branch_id,
    B_L.card_no,
    B_L.date_out,
    B_L.due_date,
    B_L.Returned_date,
    CASE
        WHEN B_L.Returned_date IS NULL OR DATEDIFF(B_L.Returned_date, B_L.

```

Late Loans Output:

Book ID	Branch ID	Card No	Date Out	Due Date	Returned Date	Days Late
5	1	567890	2022-01-05	2022-02-05	2022-02-09	4
6	2	234567	2022-01-06	2022-02-06	2022-02-10	4
7	2	890123	2022-01-07	2022-02-07	2022-03-08	29
8	3	456789	2022-01-08	2022-02-08	2022-03-10	30
14	1	565656	2022-01-14	2022-02-14	2022-03-31	45
15	3	676767	2022-01-15	2022-02-15	2022-02-21	6
16	2	787878	2022-03-05	2022-03-12	2022-03-24	12

QUERY 6A

Query description: List for every borrower the ID, name, and if there is any lateFee balance. The user has the right to search either by a borrower ID, name, part of the name, or to run the query with no filters/criteria. The amount needs to be in US dollars. For borrowers with zero (0) or NULL balance, you need to return zero dollars (\$0.00). Make sure that your query returns meaningful attribute names. In the case that the user decides not to provide any filters, order the results based on the balance amount. Make sure that you return all records.

```
# add data to late_fees table output
```

```
late_fees_cursor.execute(
```

```
"SELECT Card_No, [Borrower Name],
```

```
CASE
```

```
WHEN (LateFeeBalance = 0) THEN 0.00 ELSE LateFeeBalance
```

```
END AS LateFeeBalance
```

```
FROM vBookLoanInfo
```

```
ORDER BY LateFeeBalance DESC")
```

```
late_fees_output_records = late_fees_cursor.fetchall()
```

```
# insert record(s) into output table based on user input
```

```
for output_record in late_fees_output_records:
```

```
if (id_or_name.get() == str(output_record[0])):
```

```
late_fees_table.insert(parent = "", index = 'end', text = "",
```

```
values = (
```

```
int(output_record[0]),
```

```
output_record[1],
```

```
"$" + '{0:.2f}'.format(float(output_record[2]))
```

```

))

elif (id_or_name.get() in str(output_record[1])):

late_fees_table.insert(parent = "", index = 'end', text = "",

values = (

int(output_record[0]),

output_record[1],

"$" + '{0:.2f}'.format(float(output_record[2]))

))

```

Query 6a Borrower ID/Card No Input:

The screenshot shows a web application window titled 'LMS Application'. It has a navigation bar with tabs: Q1 (Checkout), Q2 (Add Borrower), Q3 (Add Book), Q4 (Copies Loaned), Q5 (Late Loans by Due Date), Q6a (Borrower Late Fees), and Q6b (Book Loan Info). The 'Borrower Late Fees' tab is active.

Below the navigation bar, the title 'Borrower Late Fees' is displayed. The query purpose is stated as: 'Query Purpose: List borrower ID, name, and late fee balance. Search by ID or name, or list all.' There is a text input field for 'Borrower ID or Name (Partial allowed, leave blank for all):' and a 'List Late Fees' button.

The 'Borrower Late Fees:' section shows the 'SQL Query Executed:' which is:

```

SELECT
    Card_No,
    `Borrower Name`,
    LateFeeBalance
FROM vBookLoanInfo
ORDER BY LateFeeBalance DESC;

```

Below the query, the 'Borrower Late Fees Output:' is shown as a table with three columns: Card No, Borrower Name, and Late Fee Balance. The table contains 15 rows of data, sorted by Late Fee Balance in descending order.

Card No	Borrower Name	Late Fee Balance
456789	Laura Chen	\$69.00
565656	Sophia Park	\$54.00
890123	Michael Park	\$21.75
676767	Olivia Lee	\$13.80
787878	Noah Thompson	\$9.00
567890	Tom Lee	\$4.80
234567	Emily Lee	\$3.00
222222	Rachel Lee	\$0.00
454545	Dylan Kim	\$0.00
343434	Olivia Johnson	\$0.00
121212	Chloe Park	\$0.00
989898	Olivia Smith	\$0.00
555555	Grace Hernandez	\$0.00

Query 6a Full Name Input [Screenshot]:

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Borrower Late Fees

Query Purpose: List borrower ID, name, and late fee balance. Search by ID or name, or list all.

Borrower ID or Name (Partial allowed, leave blank for all):

Laura Chen

List Late Fees

Borrower Late Fees:

SQL Query Executed:

```

SELECT
    Card_No,
    `Borrower Name`,
    LateFeeBalance
FROM vBookLoanInfo
WHERE `Borrower Name` LIKE '%Laura Chen%';

```

Borrower Late Fees Output:

Card No	Borrower Name	Late Fee Balance
456789	Laura Chen	\$69.00

In the original Book_Loans table, the borrower “John Smith” with Card_No “123456” had only one book loan, but the output shows two records for the borrower “John Smith” with a LateFeeBalance value for each record because Query 1 used the Card_No “123456” to check out a book, hence after executing Query 6a after Query 1 had been executed, the borrower “John Smith” had made two book loans where each book loan will have an associated LateFeeBalance value.

Query 6a Partial Name Input [Screenshot]:

LMS Application

Q1 (Checkout)Q2 (Add Borrower)Q3 (Add Book)Q4 (Copies Loaned)Q5 (Late Loans by Due Date)Q6a (Borrower Late Fees)Q6b (Book Loan Info)

Borrower Late Fees

Query Purpose: List borrower ID, name, and late fee balance. Search by ID or name, or list all.

Borrower ID or Name (Partial allowed, leave blank for all):

List Late Fees

Borrower Late Fees:

SQL Query Executed:

```
SELECT
    Card_No,
    `Borrower Name`,
    LateFeeBalance
FROM vBookLoanInfo
WHERE `Borrower Name` LIKE '%John%';
```

Borrower Late Fees Output:

Card No	Borrower Name	Late Fee Balance
123456	John Smith	\$0.00
333333	William Johnson	\$0.00
343434	Olivia Johnson	\$0.00
345678	Bob Johnson	\$0.00

QUERY 6B Query description: List book information in the view. The user has the right either to search by the book id, books title, part of book title, or to run the query with no filters/criteria. The late fee amount needs to be in US dollars. The late fee price amount needs to have two decimals as well as the dollar '\$' sign. For books that they do not have any late fee amount, you need to substitute the NULL value with a 'Non-Applicable' text. Make sure that your query returns meaningful attribute names. In the case that the user decides not to provide any filters, order the results based on the highest late fee remaining.

```
# add data to view_book_info table output
```

```
view_book_info_cursor.execute(
```

```
"SELECT Bk.book_id, vBLI.[Book Title], vBLI.Date_Out, vBLI.Due_Date,  
vBLI.Returned_date,
```

```
vBLI.TotalDays AS [Total Days Loaned],
```

```
vBLI.[Number of days later return] AS Days_later_return, vBLI.[Branch ID],
```

```
CASE
```

```
WHEN (vBLI.LateFeeBalance = 0) THEN 'Non-Applicable'
```

```
ELSE vBLI.LateFeeBalance END AS LateFeeBalance
```

```
FROM vBookLoanInfo AS vBLI JOIN Book AS Bk ON vBLI.[Book Title] = Bk.title
```

```
ORDER BY LateFeeBalance DESC")
```

```
view_book_info_records = view_book_info_cursor.fetchall()
```

```
# insert record(s) into output table based on user input
```

```
for output_record in view_book_info_records:
```

```
late_fee_balance = 'Non-Applicable' if (str(output_record[8]) in 'Non-Applicable')
```

```
else "$" + '{0:.2f}'.format(float(output_record[8]))
```

```
if (id_or_title.get() == str(output_record[0])):
```

```
view_book_info_table.insert(parent = "", index = 'end', text = "",
```

```
values = (  
    output_record[1], output_record[2], output_record[3],  
    output_record[4], int(output_record[5]),  
    int(output_record[6]), int(output_record[7]),  
    late_fee_balance))  
  
elif (id_or_title.get() in str(output_record[1])):  
    view_book_info_table.insert(parent = "", index = 'end', text = "",  
    values = (  
        output_record[1], output_record[2], output_record[3],  
        output_record[4], int(output_record[5]),  
        int(output_record[6]), int(output_record[7]),  
        late_fee_balance))
```

No input besides card num:

LMS Application

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Book Loan Information

Query Purpose: List book loan details for a borrower. Filter by book ID/title or list all for the borrower.

Borrower Card Number:

456789

Book ID or Title (Partial allowed, leave blank for all borrower's loans):

List Book Loans

Book Loan Info for Borrower 456789:

SQL Query Executed:

SELECT

`Book Title`,

Date_Out,

Due_Date,

Returned_date,

TotalDays,

`Number of days returned late`,

`Branch ID`,

LateFeeBalance

Book Loan Info Output:

Book Title	Date Out	Due Date	Returned Date	Total Days Loaned	Days Late	Branch ID	Late Fee Balance
Lord of the Flies	2022-01-08	2022-02-08	2022-03-10	61	30	3	\$69.00

Book ID:

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

LMS Application

Book Loan Information

Query Purpose: List book loan details for a borrower. Filter by book ID/title or list all for the borrower.

Borrower Card Number:

234567

Book ID or Title (Partial allowed, leave blank for all borrower's loans):

6

List Book Loans

Book Loan Info for Borrower 234567:

SQL Query Executed:

```

SELECT
    vBLI.`Book Title`,
    vBLI.Date_Out,
    vBLI.Due_Date,
    vBLI.Returned_date,
    vBLI.TotalDays,
    vBLI.`Number of days returned late`,
    vBLI.`Branch ID`,
    vBLI.LateFeeBalance

```

Book Loan Info Output:

Book Title	Date Out	Due Date	Returned Date	Total Days Loaned	Days Late	Branch ID	Late Fee Balance
Animal Farm	2022-01-06	2022-02-06	2022-02-10	35	4	2	\$3.00

The output shows the book with the Book Title Animal Farm, which has a book_id '6' just like the user's inputted Book ID

Query 6b Full Book Title Input [Screenshot]:

LMS Application

Q1 (Checkout)

Q2 (Add Borrower)

Q3 (Add Book)

Q4 (Copies Loaned)

Q5 (Late Loans by Due Date)

Q6a (Borrower Late Fees)

Q6b (Book Loan Info)

Book Loan Information

Query Purpose: List book loan details for a borrower. Filter by book ID/title or list all for the borrower.

Borrower Card Number:

787878

Book ID or Title (Partial allowed, leave blank for all borrower's loans):

The Hitchhiker's Guide to the Galaxy

List Book Loans

Book Loan Info for Borrower 787878:

SQL Query Executed:

SELECT
`Book Title`,
Date_Out,
Due_Date,
Returned_date,
TotalDays,
`Number of days returned late`,
`Branch ID`,
LateFeeBalance

Book Loan Info Output:

Book Title	Date Out	Due Date	Returned Date	Total Days Loaned	Days Late	Branch ID	Late Fee Balance
The Hitchhiker's Guide to th	2022-03-05	2022-03-12	2022-03-24	19	12	2	\$9.00

Partial Book Title Input [Screenshot]:

Q1 (Checkout)Q2 (Add Borrower)Q3 (Add Book)Q4 (Copies Loaned)Q5 (Late Loans by Due Date)Q6a (Borrower Late Fees)Q6b (Book Loan Info)

Book Loan Information

Query Purpose: List book loan details for a borrower. Filter by book ID/title or list all for the borrower.

Borrower Card Number:

555555

Book ID or Title (Partial allowed, leave blank for all borrower's loans):

ghts

List Book Loans

Book Loan Info for Borrower 555555:

SQL Query Executed:

```
Due_Date,
Returned_date,
TotalDays,
`Number of days returned late`,
`Branch ID`,
LateFeeBalance
FROM vBookLoanInfo
WHERE Card_No = '555555'
AND `Book Title` LIKE '%ghts%'
;
```

Book Loan Info Output:

Book Title	Date Out	Due Date	Returned Date	Total Days Loaned	Days Late	Branch ID	Late Fee Balance
Wuthering Heights	2022-02-03	2022-03-03	2022-02-18	15	0	3	Non-Applicable

Query 6b No Filters/Criteria Output [Screenshot]:

Book Loan Information

Query Purpose: List book loan details for a borrower. Filter by book ID/title or list all for the borrower.

Borrower Card Number:

Book ID or Title (Partial allowed, leave blank for all borrower's loans):

Book Loan Info for Borrower 345678:

SQL Query Executed:

```
SELECT
  `Book Title`,
  Date_Out,
  Due_Date,
  Returned_date,
  TotalDays,
  `Number of days returned late`,
  `Branch ID`,
  LateFeeBalance
```

Book Loan Info Output:

Book Title	Date Out	Due Date	Returned Date	Total Days Loaned	Days Late	Branch ID	Late Fee Balance
Pride and Prejudice	2022-01-03	2022-02-03	NULL	None	None	2	Non-Applicable

Contribution List:

FAHAD SSENTONGO collaborated closely with the team in developing database triggers and contributed significantly to features related to book and borrower data management. He also supported GUI integration and participated in query testing to ensure smooth functionality across modules.

MANAN SHAH worked hand-in-hand with the team to write and optimize SQL queries, including the creation of the vBookLoanInfo view. He actively contributed to integrating the GUI components and assisted in validating the backend logic alongside Fahad and Lam.

LAM NINH led the design and layout of the GUI while collaborating with Fahad and Manan to implement the Late Fee calculation logic. He was deeply involved in end-to-end testing and helped refine database operations and query performance through group feedback sessions.

